Lecture 6: Dynamic Programming and Value Function Iteration

Computational Bootcamp

John P. Ryan

Summer 2026

University of Wisconsin–Madison

Today's goals

1. Introduce the concept of dynamic programming generally
2. Learn to think about state variables, and add uncertainty to the growth model
3. See how characterizing the solution on paper (the endogenous grid method) beats brute force

Dynamic Programming

One model, five solvers

- You have now solved the growth model three times:
  - Lecture 2: grid search over k'
  - Lecture 4: interpolate V, optimize over k' continuously
  - Lecture 5: parallelize the loop over k
- Today we change the algorithm instead:
  - Skip work we can prove is wasted (monotonicity, concavity, Howard)
  - Delete the maximization entirely (the endogenous grid method)

Dynamic programming

- A problem is a dynamic program if it breaks into sub-problems that recur, so you can solve each one once and reuse the answer
- Fibonacci: $F(n) = F(n-1) + F(n-2)$
  - Naive recursion recomputes $F(4)$ exponentially many times
  - Store each value the first time you compute it (memoization) and runtime collapses to linear
- The growth model: "the best you can do from capital k onward" recurs at every date. Solve it once per k and iterate over the stored value function
- The whole content of a Bellman equation:

$$V(k) = \max_{k'} \underbrace{\log(c)}_{\text{static choice}} + \beta \underbrace{V(k')}_{\text{stored answer}}$$

State variables

- The state is the smallest set of variables such that, knowing them, the past does not matter
  - In the growth model: k (and z). How you got to k is irrelevant
  - Not states: your history of past consumption, anything you can reconstruct from what is already there
- Curse of dimensionality: at 100 points per state, the grid is $100^{\#\text{states}}$
  - 2 states: 10^4 points, fine. 4 states: 10^8, painful. 6 states: hopeless
- So eliminating a state variable beats any amount of micro-optimization
- Some new deep learning methods can use sparse state space (See Jesus Fernandez-Villaverde's papers and lectures)

Finite vs infinite horizon

Finite horizon (work from 25 to 65, then die): the date *is* a state

$$V_t(a) = \max_{a'} \{u(c) + \beta V_{t+1}(a')\}, \qquad V_{T+1} \equiv 0$$

- The terminal condition is known, so sweep backward once: solve T, then $T-1$, ...
- No fixed point, no convergence check. A different policy at every age — that is the point

Infinite horizon (stationary): the date drops out, one policy $g(k)$, but no terminal condition

- Instead iterate the Bellman operator: $V_{n+1} = T(V_n)$, stop when $\|V_{n+1} - V_n\|_\infty < \text{tol}$
- Legal because T is a contraction with modulus β (Lecture 3): unique fixed point, from any guess
- But convergence is geometric at rate β. $\beta = 0.99$ means *hundreds* of sweeps

Adding Uncertainty

The stochastic growth model

- Our growth model with stochastic productivity z:

$$V(k,z) = \max_{c,k'} \left\{ \log(c) + \beta \sum_{z'} \Pi(z' \mid z)\, V(k',z') \right\}, \qquad c = zk^{\alpha} + (1-\delta)k - k'$$

- The continuation value is an expectation
- Model has continuous $z \sim$ AR(1). Discretize them into a Markov chain with Tauchen or Rouwenhorst (better when ρ is near 1) in QuantEcon.jl
- z follows a discrete Markov chain, $\Pi_{ij} = P(z' = z_j \mid z = z_i)$, so rows sum to one

Making VFI faster

- Monotonicity: the policy is increasing, so if $g(k_i) = k_m$ then $g(k_{i+1}) \geq k_m$ — start each search where the last one ended
- Concavity: the objective is single-peaked in k', so stop searching once values start falling
- Howard / policy iteration: the policy converges long before the value does — next slide
- Plus your existing toolkit: optimization, interpolation, parallelism

Howard iteration

- The max is most of the cost. *Applying* a fixed policy is cheap: $T_g V = u_g + \beta Q_g V$
- Its fixed point is a linear solve, $V_g = (I - \beta Q_g)^{-1} u_g$ — that is policy iteration: Newton on the Bellman equation, 10–30 steps at any β
- Howard: don't invert, just apply T_g m times. $m = 0$ is VFI, $m = \infty$ is policy iteration, $m = 20$–50 in practice

```
# gi = policy index, ug = flow payoff, both kept from the max sweep
for _ in 1:m                        # note: no max in here
    for i_z in 1:N_z
        EV = V * Pi[i_z, :]
        for i_k in 1:N_k
            V[i_k, i_z] = ug[i_k, i_z] + beta * EV[gi[i_k, i_z]]
        end
    end
end
```

Issue: junk early g wastes the sweeps, how to choose m? Even so, you are still maximizing.

The Endogenous Grid Method

Trick 1: change the state

- VFI finds the optimum at every grid point, ignoring what we know analytically
- The usual problem, state k: the state enters through the budget constraint

$$V(k) = \max_{k'} \{u(c) + \beta V(k')\}, \qquad c = k^\alpha + (1-\delta)k - k'$$

- Same model, state cash on hand $y = k^\alpha + (1-\delta)k$ — everything available to spend today

$$V(y) = \max_{k'} \{u(c) + \beta V(y')\}, \qquad c = y - k', \qquad y' = k'^\alpha + (1-\delta)k'$$

- The nonlinearity moved off the state and onto the choice: budget is now linear, so given (c, k') today's state is one addition, $y = c + k'$, instead of inverting $k^\alpha + (1-\delta)k = c + k'$

Trick 2: use the first-order and envelope conditions

Substitute the budget constraint, so k' is the only choice: $V(y) = \max_{k'} \{u(y - k') + \beta V(y')\}$

- FOC in k'. Saving one more unit costs $u'(c)$ today and buys $\alpha k'^{\alpha-1} + 1 - \delta$ units of cash on hand tomorrow, each worth $V'(y')$:

$$\underbrace{u'(c)}_{\text{cost today}} = \beta \underbrace{\left(\alpha k'^{\alpha-1} + 1 - \delta\right)}_{\text{units tomorrow}} \times \underbrace{V'(y')}_{\text{value per unit}}$$

- Envelope. Now differentiate V in the *state*.

$$V'(y) = u'(c(y))$$

  An extra unit of cash is worth what eating it is worth: the slope of V is marginal utility

- Forward one period, $V'(y') = u'(c(y'))$. Substitute into the FOC:

$$u'\big(c(y)\big) = \beta\, h'(k')\, u'\big(c(y')\big), \qquad h(k') \equiv k'^{\alpha} + (1-\delta)k', \quad h'(k') = \alpha k'^{\alpha-1} + 1 - \delta$$

- No max, no V: this Euler equation is one equation in the policy $c(\cdot)$ alone

Trick 3: flip the grid (Carroll, 2006)

- Put the grid on k', tomorrow's capital, not on today's state y
- With k' fixed the whole right-hand side of the Euler equation is computable, and marginal utility inverts in closed form: $c = (u')^{-1}(\text{RHS})$, which under log utility is just $c = 1/\text{RHS}$
- Iterating on the consumption policy, stored as points (y_j, c_j):
  1. Fix a grid $\{k'_j\}$; precompute $y'_j = h(k'_j)$ and $h'(k'_j)$ once
  2. Use previous iteration policy function to get $c(y'_j)$ next period
  3. $\text{RHS}_j = \beta\, h'(k'_j)\, u'\big(c(y'_j)\big)$, then invert: $c_j^{new} = (u')^{-1}(\text{RHS}_j) = 1/\text{RHS}_j$
  4. Budget constraint gives today's grid point: $y_j^{new} = c_j^{new} + k'_j$ — endogenous
  5. Interpolate (y_j^{new}, c_j^{new}) to get $c(y)$ on the original grid
  6. Stop when $\max_j |c_j^{new} - c_j| < \text{tol}$
- Each iteration is arithmetic plus one interpolation. No optimizer, no root finder

Takeaways

1. The intuition: fix the choice, solve backwards for the state. VFI stands at y and searches for k'; EGM assumes k' and asks which y makes it optimal — an inversion and an addition, no search
2. Use it when the Euler equation is invertible: u' has a closed-form inverse (CRRA). Kinks, binding constraints and discrete choices need care
3. A better algorithm beats optimized code. Vectorize, exploit monotonicity, parallelize across 20 cores; EGM still wins. Upgrade the algorithm before you micro-optimize
4. Pencil & paper work speeds up computation. Our tricks reduced our computational demands by orders of magnitude.
5. You should use it. EGM is the gold standard for dynamic programming in economics.

To the code!